Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Розробка графічного редактора об’єктів»

Виконала:

студентка групи ІМ-51
Резник Валерія Вадимівна
Номер в списку - 26

Перевірив:

Рекечинський Дмитро Олександрович

Київ 2020

Мета: тримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши простий графічний редактор в об’єктно-орієнтованому стилі.

Завдання:

1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Варіант: 26

- Статичний масив обсягом 126 елементів
- “Гумовий” слід - суцільна лінія синього кольору

  Прямокутник:

  - По двом протилежним кутам
  - Чорний контур з блакитним заповненням

  Еліпс:

  - Від центру до одного з кутів охоплюючого прямокутника
  - Чорний контур з білим заповненням
- Позначка поточного типу об’єкту в меню

Вихідний код:

main.py

```
import sys
from PySide6.QtCore import Qt
from PySide6.QtGui import QAction, QPainter, QPen, QColor, QPalette,
QMouseEvent
from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,
    QWidget,
    QMenu,
    QMessageBox,
)
from shapes import Shape, Point, Line, Rectangle, Ellipse


N = 126


class DrawingArea(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)


        self.pcshape = [None] * N
        self.shape_count = 0


        self.current_shape = "Крапка"


        self.x1 = 0
        self.y1 = 0
        self.x2 = 0
```

```
        self.y2 = 0


        self.drawing = False


        self.setMinimumSize(600, 400)
        self.setAutoFillBackground(True)
        palette = self.palette()
        palette.setColor(QPalette.ColorRole.Window, QColor("white"))
        self.setPalette(palette)


    def set_shape(self, shape_name):
        self.current_shape = shape_name


    def mousePressEvent(self, event: QMouseEvent):
        if event.button() == Qt.MouseButton.LeftButton:
            self.x1 = int(event.position().x())
            self.y1 = int(event.position().y())


            self.x2 = self.x1
            self.y2 = self.y1


            self.drawing = True


            if self.current_shape == "Крапка":
                self.add_shape(Point(self.x1, self.y1))


                self.drawing = False


            self.update()
```

```
    def mouseMoveEvent(self, event: QMouseEvent):
        if self.drawing:
            self.x2 = int(event.position().x())
            self.y2 = int(event.position().y())
            self.update()

    def mouseReleaseEvent(self, event: QMouseEvent):
        if (event.button() == Qt.MouseButton.LeftButton and
self.drawing):
            self.x2 = int(event.position().x())
            self.y2 = int(event.position().y())

            if self.current_shape == "Лінія":
                shape = Line(self.x1, self.y1, self.x2, self.y2)

            elif self.current_shape == "Прямокутник":
                shape = Rectangle(self.x1, self.y1, self.x2, self.y2)

            elif self.current_shape == "Еліпс":
                shape = Ellipse(self.x1, self.y1, self.x2, self.y2)

            else: shape = None

            if shape is not None:
                self.add_shape(shape)

            self.drawing = False
            self.update()
```

```
    def add_shape(self, shape):
        if self.shape_count >= N:
            QMessageBox.warning(self, "Увага", "Досягнуто максимальної кількості фігур: 126")
            return

        self.pcshape[self.shape_count] = shape
        self.shape_count += 1

        self.update()

    def paintEvent(self, event):
        painter = QPainter(self)

        for i in range(self.shape_count):
            shape: Shape = self.pcshape[i]

            if shape is not None:
                shape.draw(painter)

        if self.drawing and self.current_shape != "Крапка":
            painter.setPen(QPen(QColor("blue"), 1, Qt.PenStyle.SolidLine))
            painter.setBrush(Qt.BrushStyle.NoBrush)

            if self.current_shape == "Лінія":
                painter.drawLine(self.x1, self.y1, self.x2, self.y2)

            elif self.current_shape == "Прямокутник":
```

```
                left = min(self.x1, self.x2)
                top = min(self.y1, self.y2)
                width = abs(self.x2 - self.x1)
                height = abs(self.y2 - self.y1)

                painter.drawRect(left, top, width, height)

            elif self.current_shape == "Еліпс":
                radius_x = abs(self.x2 - self.x1)
                radius_y = abs(self.y2 - self.y1)

                painter.drawEllipse(
                    self.x1 - radius_x,
                    self.y1 - radius_y,
                    radius_x * 2,
                    radius_y * 2,
                )

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Lab2")
        self.resize(800, 600)

        self.drawing_area = DrawingArea(self)
        self.setCentralWidget(self.drawing_area)

        self.create_menu()
```

```
    def create_menu(self):
        menu_bar = self.menuBar()

        file_menu = menu_bar.addMenu("Файл")
        clear_action = QAction("Очистити", self)
        clear_action.triggered.connect(self.clear_canvas)
        file_menu.addAction(clear_action)

        self.objects_menu = QMenu("Об'єкти", self)
        menu_bar.addMenu(self.objects_menu)

        shape_names = ["Крапка", "Лінія", "Прямокутник", "Еліпс"]

        self.shape_actions: dict[str, QAction] = {}

        for name in shape_names:
            action = QAction(name, self)
            action.setCheckable(True)
            action.triggered.connect(lambda checked=False, n=name:
self.select_shape(n))

            self.objects_menu.addAction(action)
            self.shape_actions[name] = action

        self.shape_actions["Крапка"].setChecked(True)

        self.objects_menu.aboutToShow.connect(self.update_objects_menu)
```

```
        help_menu = menu_bar.addMenu("Довідка")
        about_action = QAction("Про програму", self)
        about_action.triggered.connect(self.show_about)
        help_menu.addAction(about_action)

    def select_shape(self, shape_name):
        self.drawing_area.set_shape(shape_name)

        for name, action in self.shape_actions.items():
            action.setChecked(name == shape_name)

    def update_objects_menu(self):
        for name, action in self.shape_actions.items():
            action.setChecked(name == self.drawing_area.current_shape)

    def clear_canvas(self):
        self.drawing_area.pcshape = [None] * N
        self.drawing_area.shape_count = 0
        self.drawing_area.update()

    def show_about(self):
        QMessageBox.about(self, "Про програму", "Lab2 - графічний
редактор\n")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
```

```
    sys.exit(app.exec())
```

shapes.py

```
from abc import ABC, abstractmethod
from PySide6.QtCore import QPoint
from PySide6.QtGui import QColor, QPainter, QPen, QBrush


class Shape(ABC):
    def __init__(self, x1: int, y1: int, x2: int, y2: int):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    @abstractmethod
    def draw(self, painter: QPainter):
        pass


class Point(Shape):
    def __init__(self, x: int, y: int):
        super().__init__(x, y, x, y)

    def draw(self, painter: QPainter):
        painter.setPen(QPen(QColor("black"), 3))
        painter.drawPoint(QPoint(self.x1, self.y1))


class Line(Shape):
    def draw(self, painter: QPainter):
```

```
        painter.setPen(QPen(QColor("black"), 2))
        painter.drawLine(self.x1, self.y1, self.x2, self.y2)


class Rectangle(Shape):
    def draw(self, painter: QPainter):
        left = min(self.x1, self.x2)
        top = min(self.y1, self.y2)
        width = abs(self.x2 - self.x1)
        height = abs(self.y2 - self.y1)

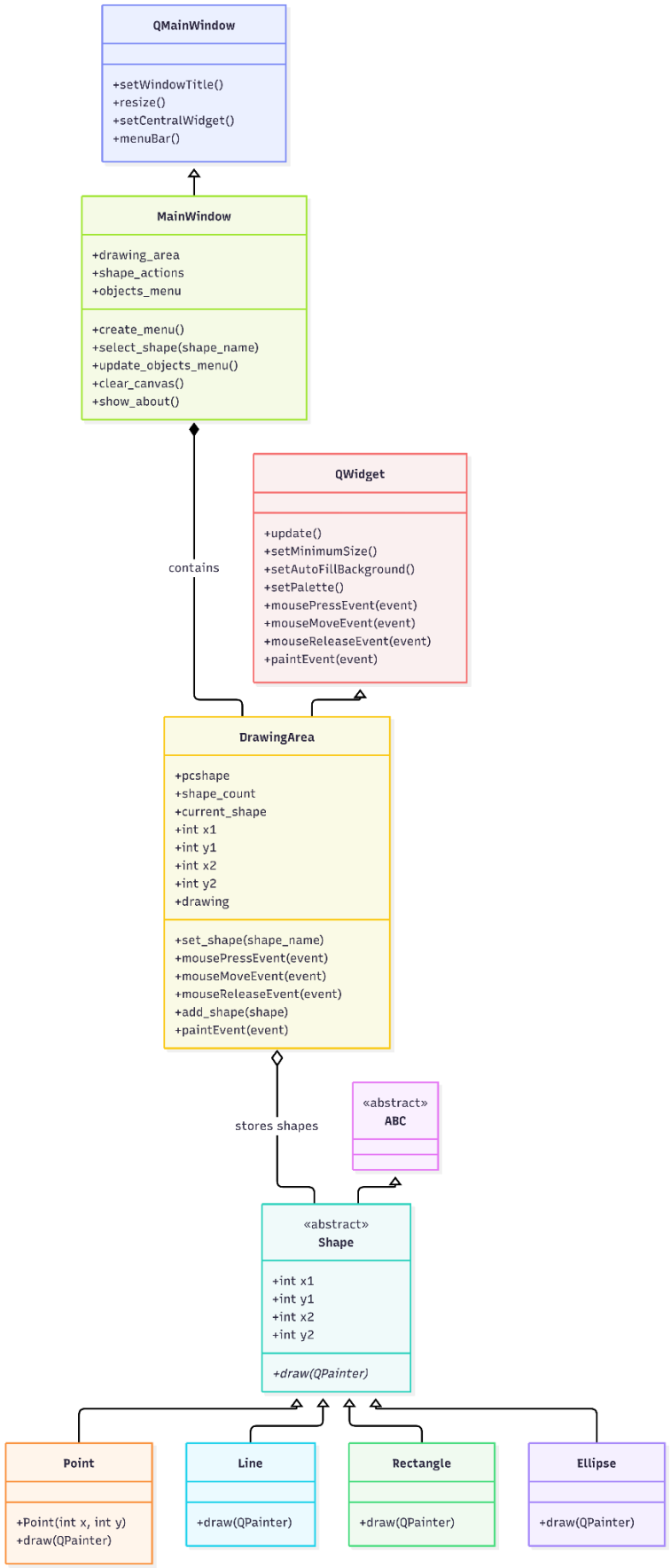
        painter.setPen(QPen(QColor("black"), 2))
        painter.setBrush(QBrush(QColor("#87CEFA")))
        painter.drawRect(left, top, width, height)


class Ellipse(Shape):
    def draw(self, painter: QPainter):
        radius_x = abs(self.x2 - self.x1)
        radius_y = abs(self.y2 - self.y1)

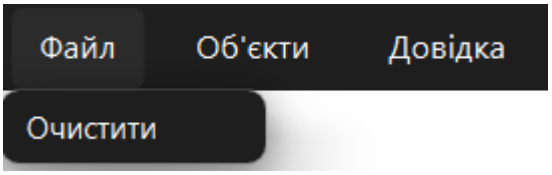
        left = self.x1 - radius_x
        top = self.y1 - radius_y
        width = radius_x * 2
        height = radius_y * 2

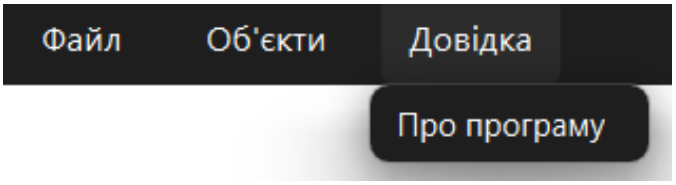
        painter.setPen(QPen(QColor("black"), 2))
        painter.setBrush(QBrush(QColor("white")))
        painter.drawEllipse(left, top, width, height)
```

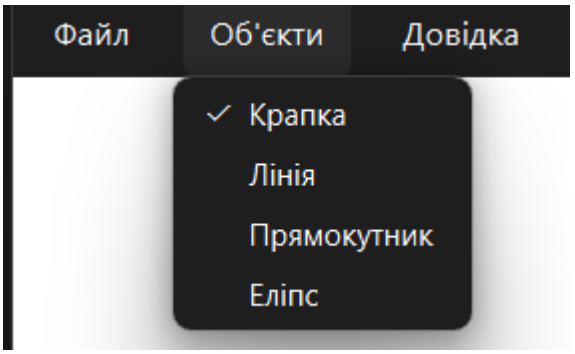
Схема успадкування класів:

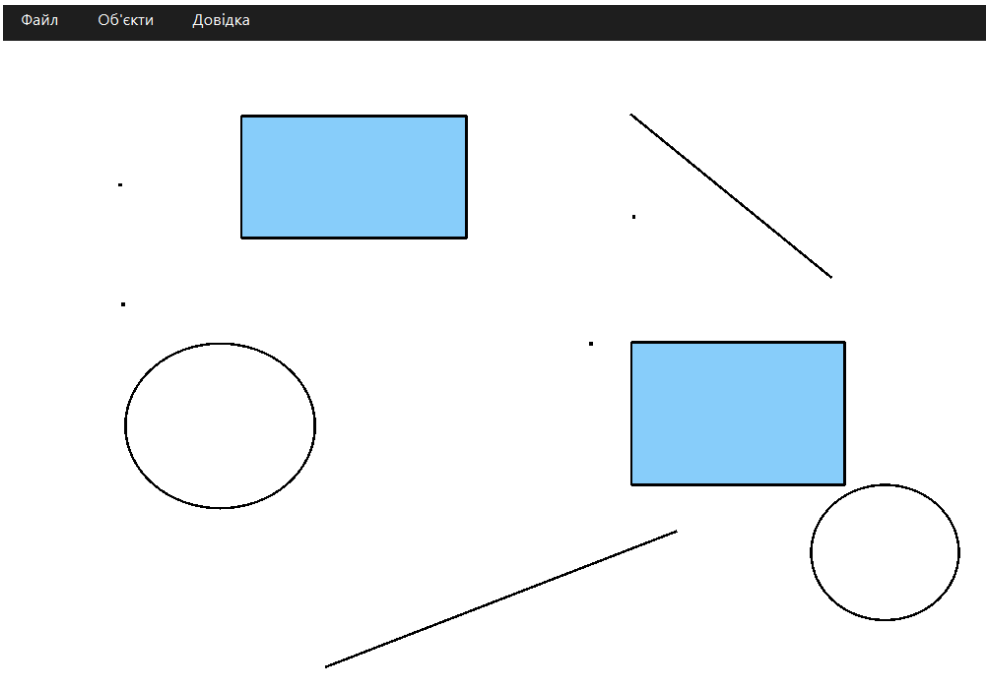


Тестування (скріншоти):









Висновок:

Під час виконання лабораторної роботи було розроблено простий графічний редактор в об’єктно-орієнтованому стилі. Було закріплено навички використання абстракції, інкапсуляції, успадкування та поліморфізму. Створено базовий абстрактний клас Shape та класи-нащадки для малювання крапки, лінії, прямокутника й еліпса. Також реалізовано взаємодію з користувачем через меню та обробку подій миші. У результаті роботи було поглиблено розуміння принципів об’єктно-орієнтованого програмування та їх практичного застосування.